

Generación de extremo a extremo sin escribir la app a mano

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 5 de 12 · Notas en español (es-ES)

La sesión 5 terminó el stack de gestión de tareas que la 4 había pausado a mitad de frontend: reanudar la misma conversación para que el orquestador siga la lista de to-dos, arrancar backend y frontend, ejercitar Swagger y la UI Kanban, y extender el repo con skills con forma de Docker/CI — todavía sin que el formador teclee código de aplicación. «Sin escribir la app a mano» no significa «sin criterio de ingeniería». Significa que *el origen de handlers y componentes es el bucle del agente*, dirigido por CLAUDE.md, skills, OpenAPI, Figma/MCP y los modos de permiso/plan.

VUESTRA UI GENERADA NO COINCIDIRÁ CON LA GRABACIÓN

Puertos (3000 / 5173), Vite, Tailwind, Playwright y los prompts de Docker Desktop son del momento del demo. Si Claude abre Docker Desktop sin que se lo pidáis, apretad la skill. Preferid el comportamiento vigente de **Claude Code** frente a los clics exactos de este recorrido.

Reanudar es una función, no un prompt nuevo

Como la sesión anterior se detuvo en la verificación, el formador se quedó en la *misma conversación* y pidió continuar la generación. El bucle agentic ya tenía la lista de to-dos; los ítems restantes (páginas, routers, cableado) siguieron. Solo funciona si el contexto sigue ahí: una sesión nueva exigiría reafirmar artefactos (ruta OpenAPI, URL de Figma, skills). El modo de permiso sigue aplicando a cada llamada a tool restante.

Ejecutar lo generado

Un prompt corto — «run my backend and frontend application» — basta si existen scripts en `package.json`; el agente los lee en vez de que peguéis conjuros npm. Documentad arranque/parada en el README para que Claude no invente comandos. En el laboratorio: docs de API en localhost:3000, SPA en 5173. Registro/login, crear una tarea («learn Claude»), mover columnas. Faltar drag-and-drop era un requisito ausente, no un fallo del modelo: podéis añadirlo después con una captura más el nombre de la ruta para ahorrar tokens.

- Swagger: servidor en local, obtener un bearer, llamar rutas con candado (usuario actual, tareas).
- Hot reload: si el frontend es un dev server estilo Vite, las ediciones guiadas por CLAUDE.md (filtros de tres filas a una) aparecen sin rebuild completo.
- Ignorad u omitid pasos de verificación cuando se coman un proyecto pequeño (teatro de rate-limit de auth); decidlo explícitamente.
- Las palabras humanizer («cooking», «marinating») en la UI son loaders, no vocabulario extra facturable ni otro modelo.

Los deberes antes de generar ganan a los prompts gigantes

Pedro preguntó si las instrucciones por carpeta se pelean en soluciones de varias capas. Vijay: mantened una regla global de aclarar en conflicto; poned convenciones de nombres y patrones de diseño en la capa que los posee (camelCase Java en el CLAUDE.md de backend, reglas CSS bajo una nota en public/). Las plantillas de controladores (inyectar ILogger, mediator, ...) valen si sois precisos. Sobre-especificar cada patrón GoF dispara iteraciones y coste: los ficheros de la cohorte no iban tan al fondo a propósito.

Skills de contenedor y pipeline: seguís sin escribir la app a mano

Cuando la app corrió, el formador generó Dockerfiles vía skills (imagen frontend estilo nginx, imagen backend con notas de env/db) y habló de un **mono repo** con frontend, backend, Terraform, Kubernetes, Jenkins o GitHub Actions — cada uno con su skill. Pedid a Claude que escriba el SKILL.md si queréis, y luego haced slash con una tarea estrecha («solo el contenedor frontend»). Verificad .gitignore para que node_modules no llegue a GitHub. Sigue siendo entrega de extremo a extremo hecha por agentes, no por vosotros autorando JavaScript.

Qué significaba «production-ready» en la sala

El eslogan de sesiones anteriores — apps listas para producción sin una línea de código — es una afirmación de aula sobre *quién teclea la implementación*. Seguís revisando diffs, secretos, deriva de OpenAPI, accesibilidad y si Playwright o Selenium deben estar en el plan. Los tests que el agente corre en verify pueden ni siquiera ser ficheros que conservéis; el formador cazó validación sin módulo de test y señaló que conviene capturar esos casos como tests reales para no pagar a regenerarlos después.

NO CONFUNDÁIS EL CÓDIGO DEL EXAMEN CON ESTE LABORATORIO

Los escenarios escritos de Foundation no son «abrid el IDE y generad la app». Piden razonar sobre plan frente a execute, CLAUDE.md frente a skills, MCP frente a adjuntos, y adjuntos conscientes de tokens. Este laboratorio es cómo se sienten esas piezas montadas.

Puente hacia hooks (módulo siguiente)

La sesión cerró nombrando los siguientes conceptos de Claude Code: **subagentes** (ya presentados), **hooks** y **plugins**. La generación de extremo a extremo que formatea código pidiendo al modelo «sigue Prettier en CLAUDE.md» es el camino caro. El módulo 06 mueve ese trabajo a hooks locales posteriores a la tool para que la misma app greenfield cueste cientos o miles de tokens menos en el reprocesado.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Ejecutar, reanudar y generación dirigida por skills en el cliente de código.

- **Inicio de Claude Docs**

<https://docs.claude.com/en/home>

Comportamiento de producto si se ha movido la vista previa del navegador en el IDE o Docker.

- **Límites de tasa**

<https://docs.claude.com/en/api/rate-limits>

Los bucles largos de verify y los medidores de uso explican por qué se saltan comprobaciones redundantes.

- **Anthropic Academy (Skilljar)**

<https://anthropic.skilljar.com>

Cursos oficiales; este laboratorio es práctica UST, no un ejercicio de Skilljar.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.